

Syllabus



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

BCO232A	Software Engineering and Project Management	3-0-0 [3]
UNIT 1	Introduction- Introduction to Software Engineering, Software Components, Software Characteristics, Software Crisis, Engineering aspects of Software production – necessity of automation .Job responsibilities of Programmers and Software Engineers as Software developers. Software Development Life Cycle (SDLC)	
UNIT 2	Process Models and Program Design Techniques- Software Development Process Models – Code & Fix model, Waterfall model, Incremental model, Rapid Prototyping model, Spiral (Evolutionary) model. Software Requirement Specifications (SRS), Management of User Needs, Data Flow Diagrams, Entity Relationship Diagrams, Decision Tables, SRS Document, Design Techniques – Structured Programming, Coupling and Cohesion, Abstraction and Information Hiding, Software Modeling Tools –Data flow Diagrams, UML and XML.	
UNIT 3	Software Testing: Testing Objectives, Unit Testing, Integration Testing, Acceptance Testing, Regression Testing, Verification and Validation: Testing of Software Products – Black-Box Testing and White-Box Testing, Static Analysis, Symbolic Execution and Control Flow Graphs –Cyclomatic Complexity. Maintenance and its need and types of maintenance. CASE tools & graphical reporting tools.	
UNIT 4	Project Management: project, project specification parameters, principle &life cycle of project management, project management Plan, why the project is delayed? and scheduling activities, Task Network & scheduling methods, critical Path, PERT& CPM. Effort Estimation Techniques Monitoring & Control: Change Control, Software Configuration Management (SCM). Risk Analysis and Management, RMMM, Software Metrics.	
UNIT 5	Quality Management and People Management- Introduction, Understanding Behavior, Organizational Behavior, Selecting The Right Person For The Job, Motivation, The Old man – Hackman Job Characteristics Model , Working in Groups, Organization and team structures, Decision Making, Leadership, Organizational Structures, Stress, Health And Safety.	

Mapping of CO with PO



JECRC
UNIVERSITY
BUILD YOUR WORLD

Course Outcome	Program Outcome												Program Specific Outcome		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	L		H		M						L				
CO2		H		H								M	M		
CO3								H	M	L				H	
CO4	L		L			L	M								L

DEFINITION OF SOFTWARE PROCESS



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- A **framework** for the activities, actions, and tasks that are required to build high-quality software.
- SP defines the approach that is taken as software is engineered.
- Is not equal to software engineering, which also encompasses **technologies** that populate the process— technical methods and automated tools.

PROCESS MODEL



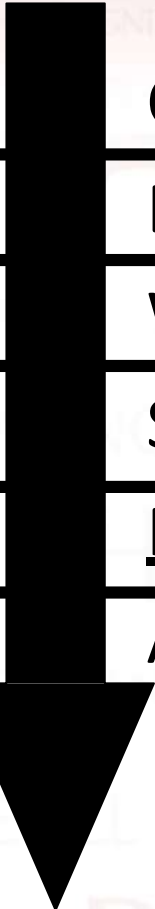
JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- A *process model* is a formal way of representing how a business operates
- *Data flow diagramming* shows business processes and the data that flows between them
- *Logical process* models describe processes without suggesting how they are conducted
- *Physical models* include information about how the processes are implemented

Timeline of Methodologies



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD



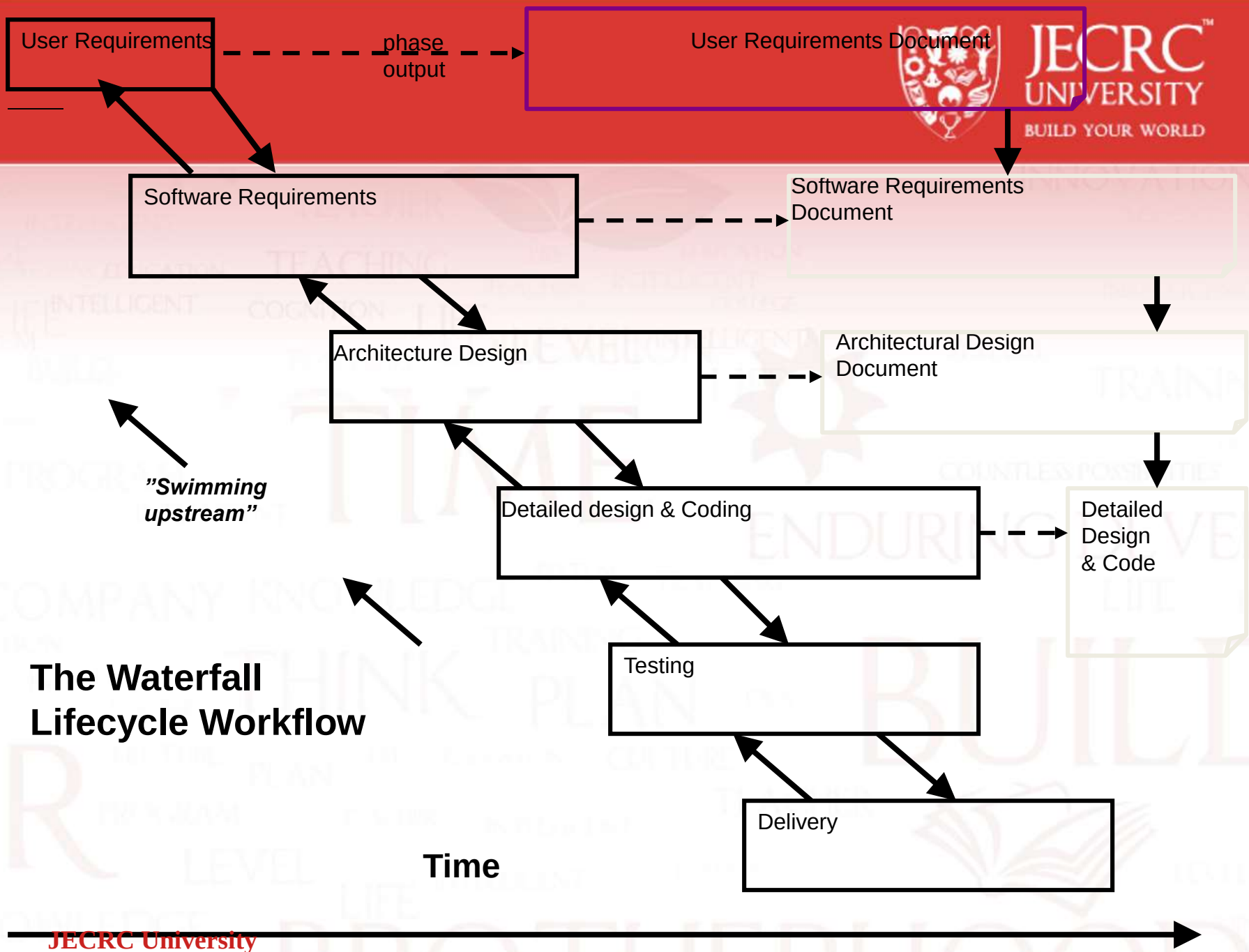
1950s	Code & Fix
1960s	Design-Code-Test-Maintain
1970s	Waterfall Model
1980s	Spiral Model
1990s	<u>R</u> apid <u>A</u> pplication <u>D</u> evelopment, V Model
2000s	Agile Methods

1. The Waterfall Model



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- The waterfall model is the classic lifecycle model – it is widely known, understood and (commonly?) used.
- In some respect, waterfall is the “common sense” approach.
- Introduced by Royce 1970.



Advantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

1. Easy to understand and implement.
2. Widely used and known (in theory!)
3. Reinforces good habits: define-before- design, design-before-code
4. Identifies deliverables and milestones
5. Document driven, URD, SRD, ... etc. Published documentation standards, e.g. PSS-05.
6. Works well on mature products and weak teams.

Disadvantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

1. Idealised, doesn't match reality well.
2. Doesn't reflect iterative nature of exploratory development.
3. Unrealistic to expect accurate requirements so early in project
4. Software is delivered late in project, delays discovery of serious errors.
5. Difficult to integrate risk management

2. Code-and-Fix Model



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

This model starts with an informal general product idea and just develops code until a product is "ready" (or money or time runs out).

Work is in random order.

Corresponds with no plan! (**Hacking!**)

Advantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

1. No administrative overhead
2. Signs of progress (code) early.
3. Low expertise, anyone can use it!
4. Useful for small “***proof of concept***” projects,
e.g. as part of risk reduction.

Disadvantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

1. Dangerous!
 1. No visibility/control
 2. No resource planning
 3. No deadlines
 4. Mistakes hard to detect/correct
2. Impossible for large projects, communication breakdown, chaos.

3. Spiral Model



Since end-user requirements are hard to obtain/define, it is natural to develop software in an ***experimental*** way: e.g.

1. Build some software
2. See if it meets customer requirements
3. If no goto 1 else stop.

This loop approach gives rise to structured **iterative lifecycle models**.

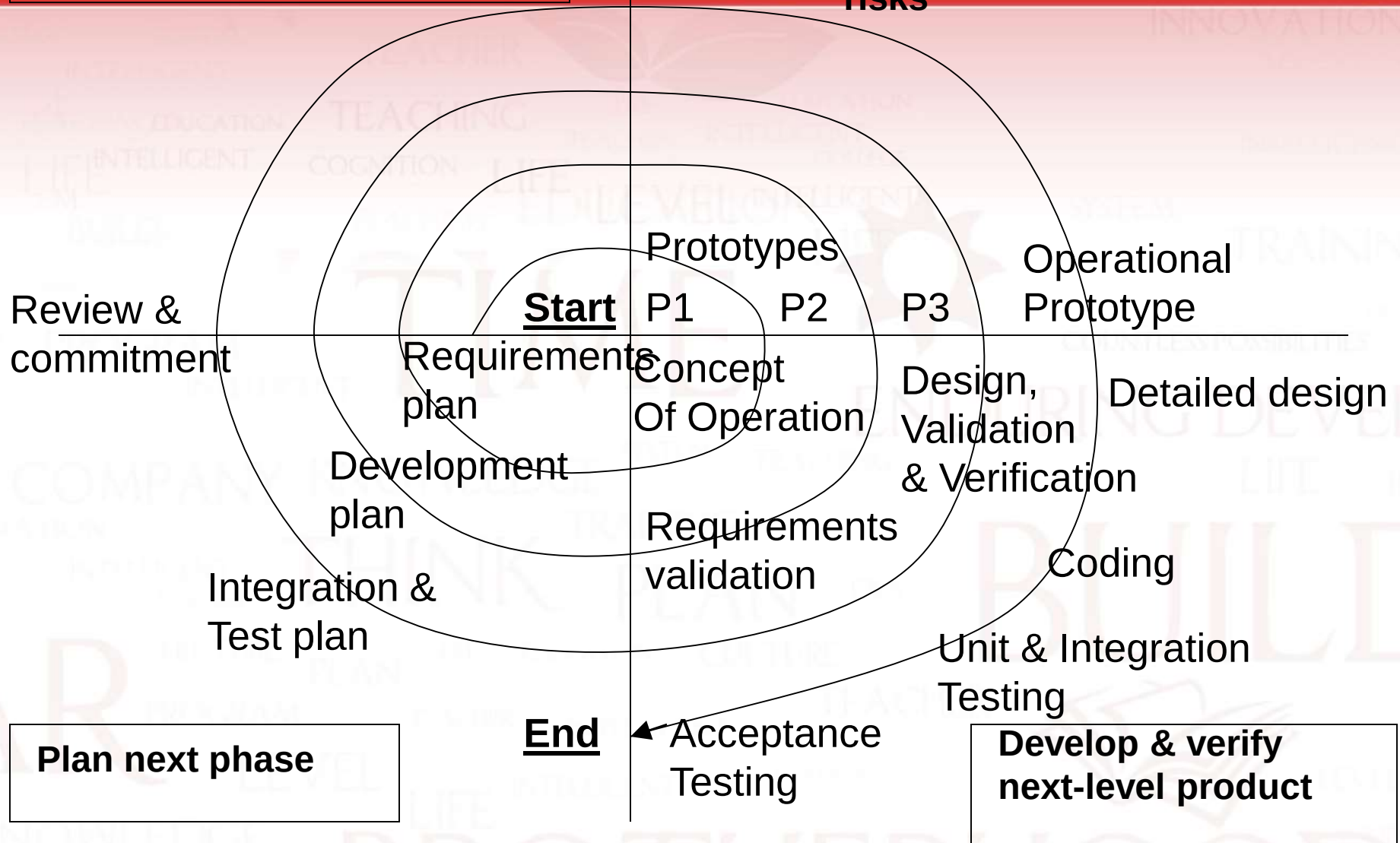
In 1988 Boehm developed the spiral model as an iterative model which includes ***risk analysis*** and ***risk management***.

Key idea: on each iteration identify and solve the sub-problems with the ***highest risk***.

Cumulative cost

**Determine objectives,
alternatives & constraints**

**Evaluate alternatives,
Identify & resolve
risks**





Each cycle follows a waterfall model by:

1. Determining objectives
2. Specifying constraints
3. Generating alternatives
4. Identifying risks
5. Resolving risks
6. Developing next-level product
7. Planning next cycle

Advantages



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

1. Realism: the model accurately reflects the iterative nature of software development on projects with unclear requirements
2. Flexible: incorporates the advantages of the waterfall and rapid prototyping methods
3. Comprehensive model decreases risk
4. Good project visibility.

Disadvantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Needs technical expertise in risk analysis to really work
- Model is poorly understood by non-technical management, hence not so widely used
- Complicated model, needs competent professional management. High administrative overhead.

4. Rapid Prototyping



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

Key idea: Customers are non-technical and usually don't know what they want/can have.

Rapid prototyping emphasises requirements analysis and validation, also called:

- ***customer oriented development,***
- ***evolutionary prototyping***

Requirements Capture



JECRC
UNIVERSITY
BUILD YOUR WORLD

Quick Design

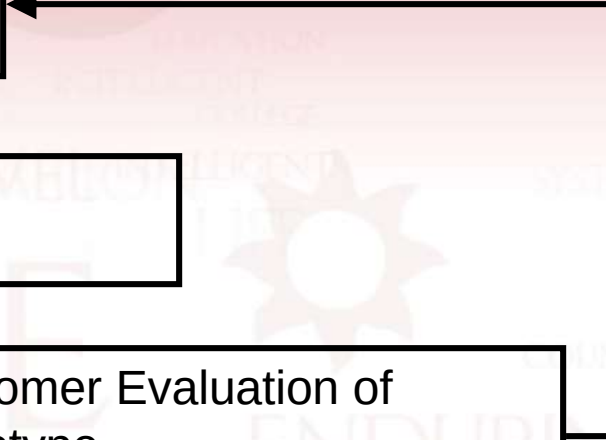
Iterate

Build Prototype

Customer Evaluation of
Prototype

Engineer Final
Product

The Rapid Prototype Workflow



Advantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

1. Reduces risk of incorrect user requirements
2. Good where requirements are changing/uncommitted
3. Regular visible progress aids management
4. Supports early product marketing

Disadvantages



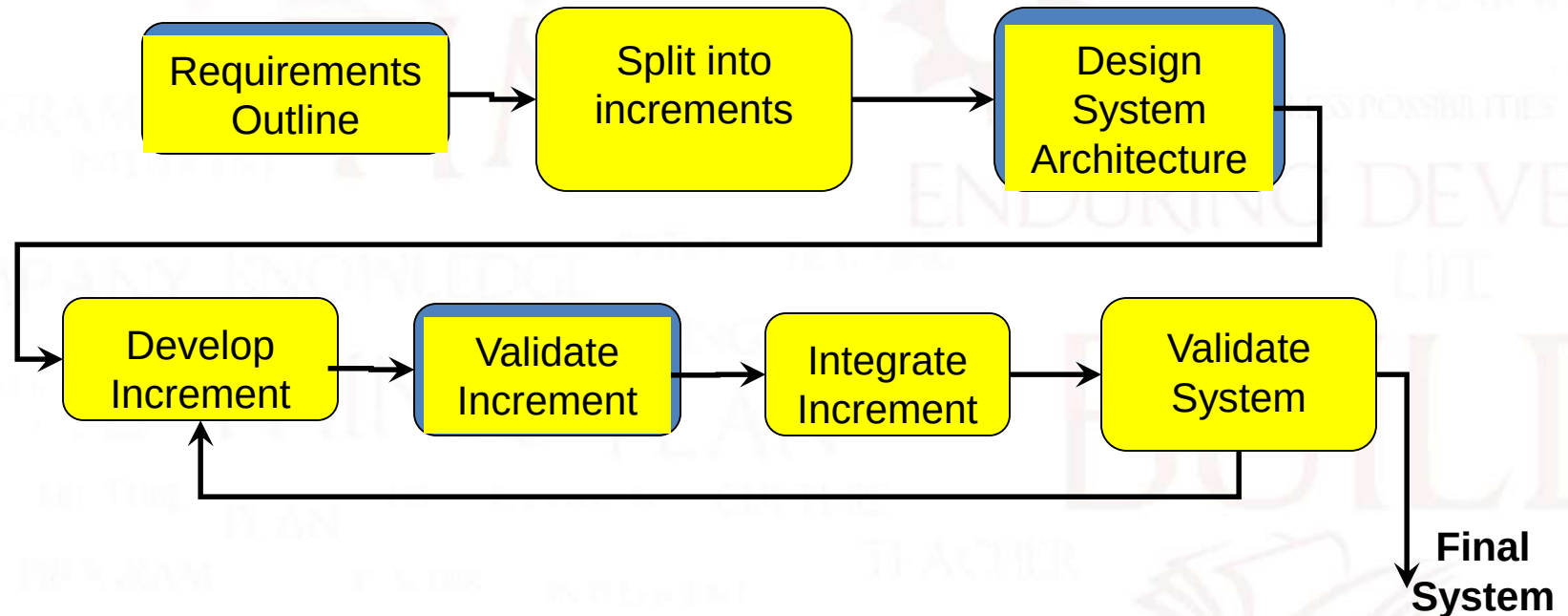
JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

1. An unstable/badly implemented prototype often becomes the final product.
2. Requires extensive customer collaboration
 - Costs customers money
 - Needs committed customers
 - Difficult to finish if customer withdraws
 - May be too customer specific, no broad market
3. Difficult to know how long project will last
4. Easy to fall back into code-and-fix without proper requirements analysis, design, customer evaluation and feedback.

5. Incremental Model



- Combine the advantages of Waterfall and Evolutionary Model.

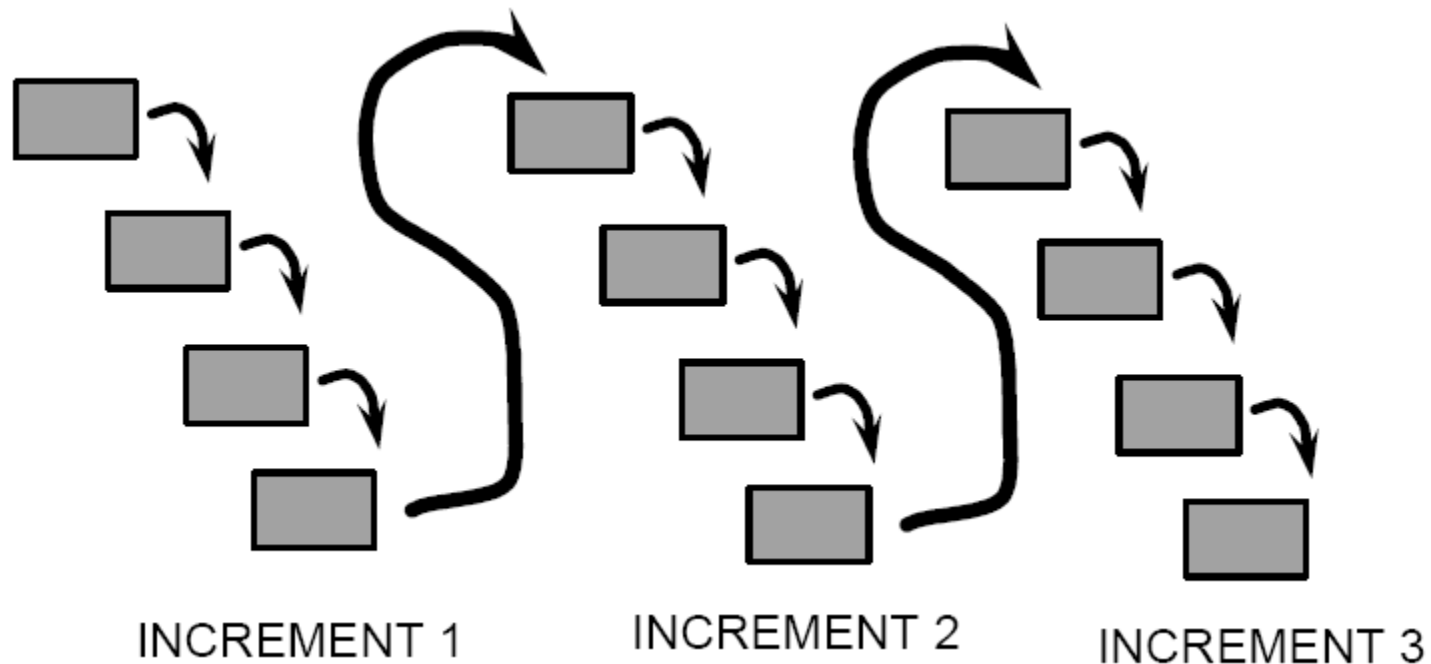


Cont...



JECRC
UNIVERSITY
BUILD YOUR WORLD

- Each increment is a mini-waterfall.



Cont...



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Prioritizes the services to be provided by the system.
- Maps these requirements to *Increment* based on priority.
- Freezes requirement for the current *Increment*.

Requirements for the later increments can evolve concurrently.

- Each *Increment* release is a working system:
 - Allows user to experiment.
 - Can be put into service right away.

Advantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Early utilization:
the 1st increment satisfies the most critical requirement.
- Early increments can serve as prototypes.
- Lower risk of overall project failure.
- Most crucial and basic services are implemented first
→ receives multiple testing throughout development.

Disadvantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Hard to map requirement into small increments (< 20,000 lines of code).
- Hard to define the basic services that are shared by all subsequent increments.
- Popular variant:
AGILE method:
 - eXtreme Programming (XP)
 - Not covered here.

6. RAD Model



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

WHAT IS RAD ???

- RAD model is Rapid Application Development model.
- It is a type of Incremental model.
- In RAD the Components are developed in parallel Manner.
- It is a faster software development process.

PHASES IN RAD



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Requirements Planning phase
- User design phase
- Construction phase
- Cutover phase

1. Requirements Planning phase



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

- Users, managers, and IT staff members discuss on Business needs.
- They discuss on System requirements.
- They also discuss on Project scope
- It decide who will generate software.
- It tells what software will do.

2. User design phase



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

- It is also called as Modeling phase.
- User Design phase is a continuous interactive process.
- During this phase, users interact with software model.
- It allows users to understand, modify the System.
- It approve a working model of the system that meets
 - their needs.

3. Construction phase



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Focuses on program and application development task.
- Tasks are
 - * Programming and application development,
 - * Coding,
 - * Unit-integration and
 - * System testing.

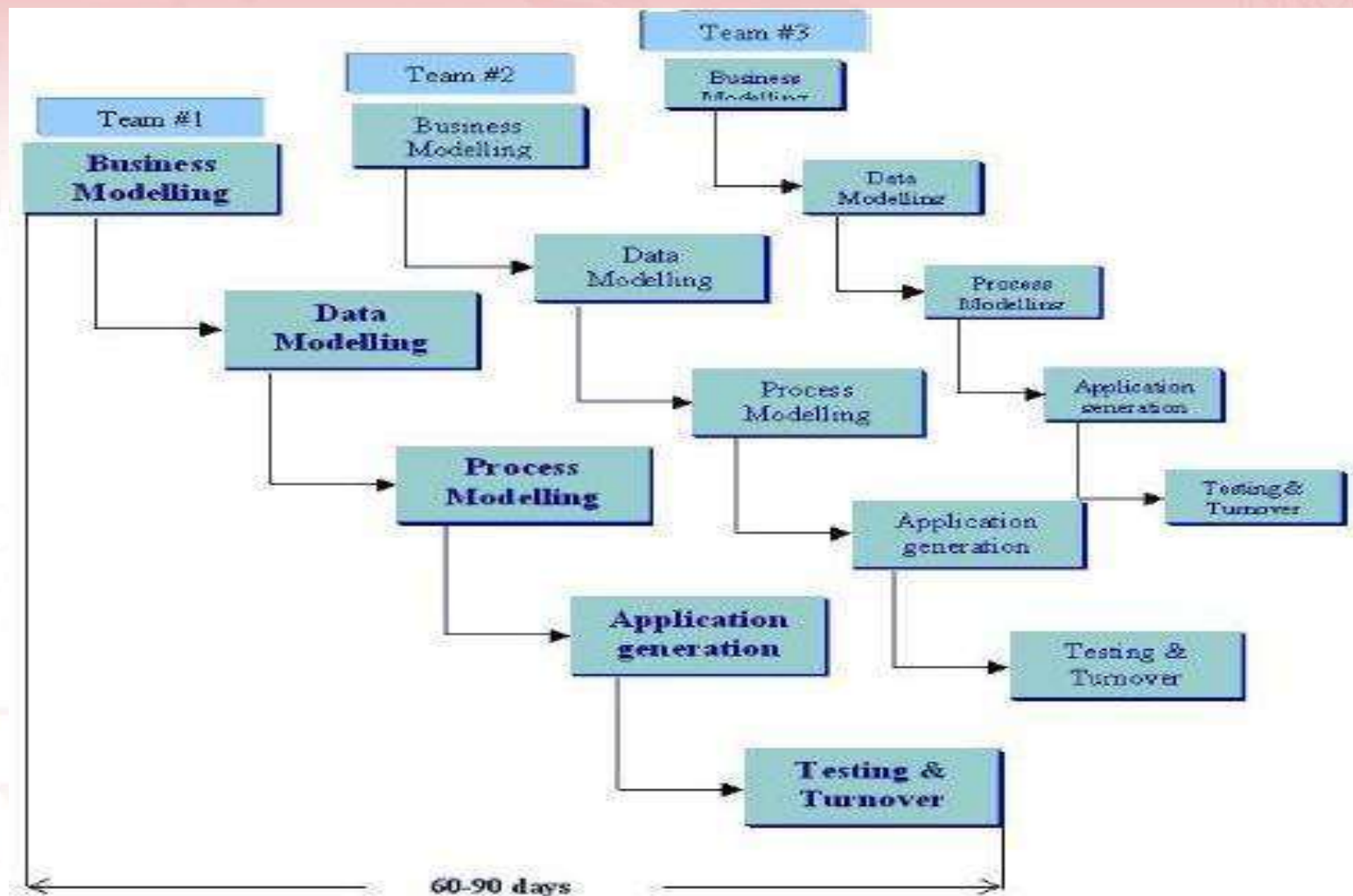
4. Cutover phase



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- It is the final (Phase) tasks in the System Development Life Cycle (SDLC).
- Its tasks are
 - * Data conversion,
 - * Full-scale testing,
 - * System change over,
 - * User training.
- In this phase the new system is built, and delivered.

RAD Model Diagram



Advantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- RAD reduces the development time.
- Increases reusability of components.
- Greater Customer Satisfaction.
- Faster Delivery Time.
- Simple and Better Quality.

Disadvantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Requires highly skilled developers/designers.
- RAD is not appropriate when technical risk are high.
- Cant use for small projects.
- Absence of reusable component can lead to failure of the project.

7. Agile Model



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Agile model is a combination of iterative and incremental process model.
- focus on process adaptability and customer satisfaction.
- It is intended to be a collection of values, principles, and practices for modeling software.
- Agile works by breaking down the project into smaller chunks and then continuously delivering them in short two weeks cycles called iterations





Agile Manifesto



Individuals and interactions over
processes and tools

Working software over **comprehensive**
documentation



Customer collaboration over
contract negotiation



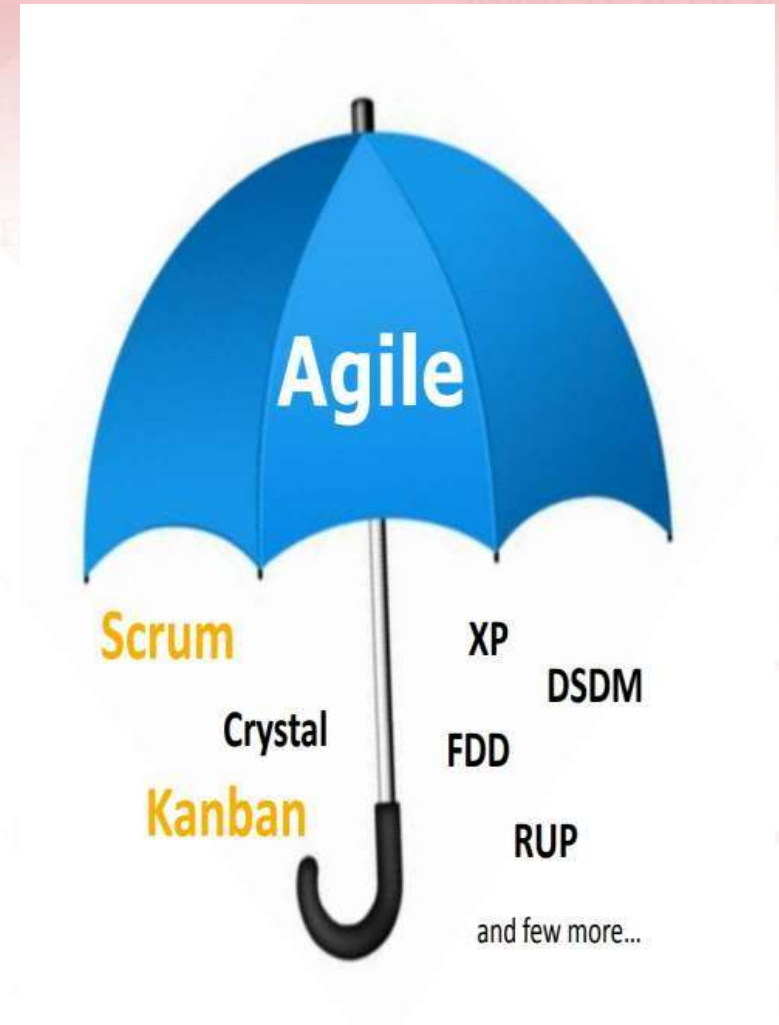
Responding to change over
following a plan



The main agile methodologies that are being used today are:-

Extreme Programming (XP) and SCRUM.

- **Extreme Programming (XP)** is the coding of what the customer specifies, and the testing of that code. Extreme programming is a software development methodology intended to improve software quality. It is a powerful tool when the customer is not sure about the functionality of the system.
- **Scrum** consists of Scrum Teams and their associated roles, events, artifacts, and rules.



Advantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

Advantages of Agile model:

- Customer satisfaction by rapid, continuous delivery of useful software.
- People and interactions are emphasized rather than process and tools.
- Customers, developers and testers constantly interact with each other.
- Working software is delivered frequently (weeks rather than months).

Disadvantages



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

Disadvantages of Agile model:

- In case of some software deliverables, especially the large ones, it is difficult to assess the effort required at the beginning of the software development life cycle.
- There is lack of emphasis on necessary designing and documentation.
- The project can easily get taken off track if the customer representative is not clear what final outcome that they want.
- Only senior programmers are capable of taking the kind of decisions required during the development process. Hence it has no place for newbie programmers, unless combined with experienced resources.

DATA FLOW DIAGRAM



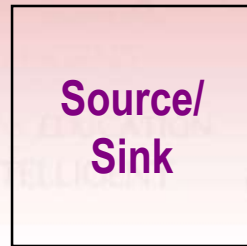
JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

A **data flow diagram (DFD)** maps out the **flow** of information for any process or system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show **data** inputs, outputs, storage points and the routes between each destination.

Data Flow Diagrams Symbols



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD



System – a group of interrelated procedures used for a business function, with an identifiable boundary, working together for some purpose.

Analysis – separation of a whole into its component parts

Design – to create, fashion, execute, or construct according to plan

Physical Data Flow Diagrams – show how the current system flows

Logical Data Flow Diagrams – show the data flow, structure, and requirements of a new system

Data Flow Diagrams Symbols



JECRCTM
UNIVERSITY
BUILD YOUR WORLD



Source/Sink – help to establish the boundaries of the system. A source identifies the origin of data inflow to the system. A sink identifies the outflow of a system, many times as information.

Sometimes referred to an entity, a source may be a customer, vendor, employee, or even another system. A single entity can be both a source and a sink.

Data Flow Diagrams Symbols



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

**Source/
Sink**



DATA STORE



Data Flow Lines

Processes – are the activities (manual and automated) that transform the inputs, transport data from process to process, stores the data, and produce the outputs of a system.

Processes are used on every DFD starting with an over all process on the context level diagram, the system. The system is then decomposed until a primitive level is obtained. The primitive level is the point in which the relevant activities of a process are identified.

Data Flow Diagrams Symbols



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

**Source/
Sink**

**0.0
Process**

DATA STORE

Data Flow Lines

Data Store – is the resting place of the data in a system. A data store can be in the form of paper, a disk file or any other media.

Normally the word 'data' does **not** appear in the title of a data store. Some examples of data stores are Customer Order, Payment, Invoice, Time Card.....

Data Flow Diagrams Symbols



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

**Source/
Sink**

**0.0
Process**

DATA STORE

Data Flow Lines

Data Flow – is the data in motion. Data can move from the outside (source) into a process. Once the inside of a system data must flow from place to place through a process, the flow lines show this movement.

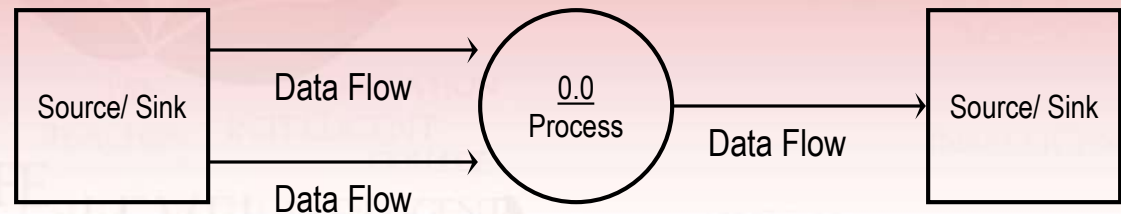
The lines are labeled to provide clarity and meaning to the data moving through the system.

Data Flow Diagrams Levels

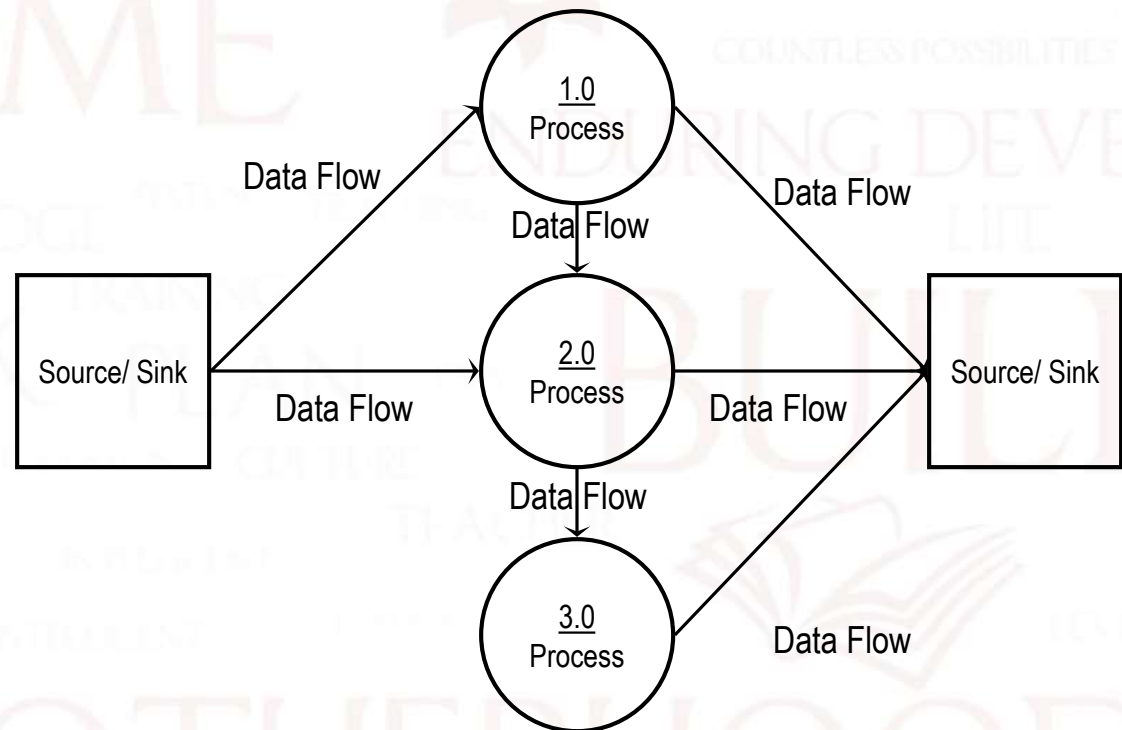


JECRC
UNIVERSITY
BUILD YOUR WORLD

Context Level DFD



Level 0 DFD



**Source/
Sink**

**0.0
Process**

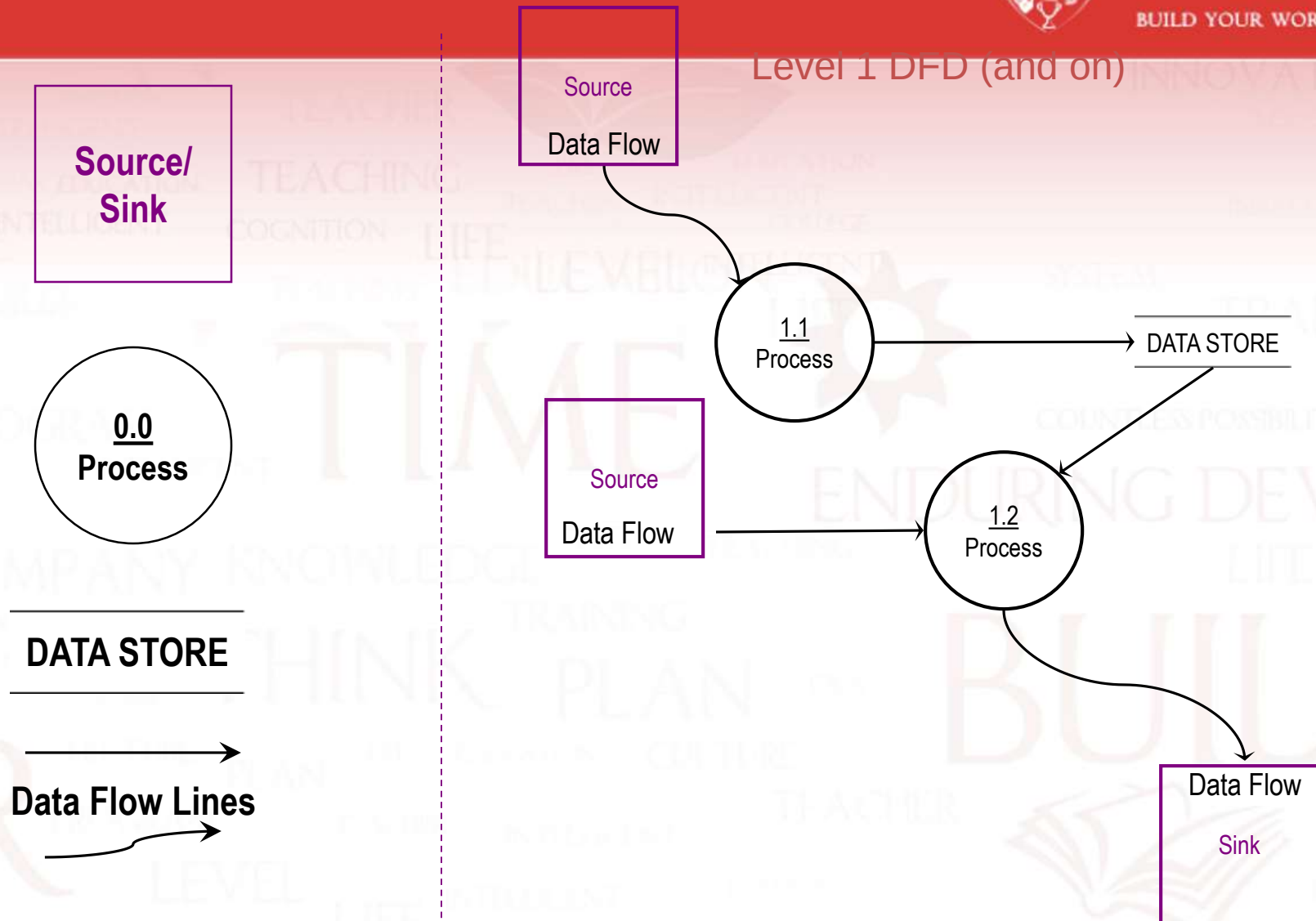
DATA STORE

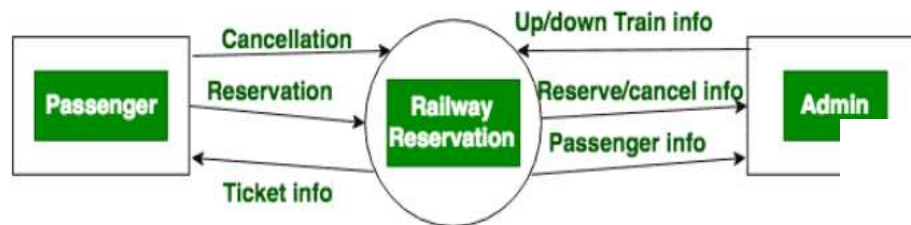
Data Flow Lines

Data Flow Diagrams Levels

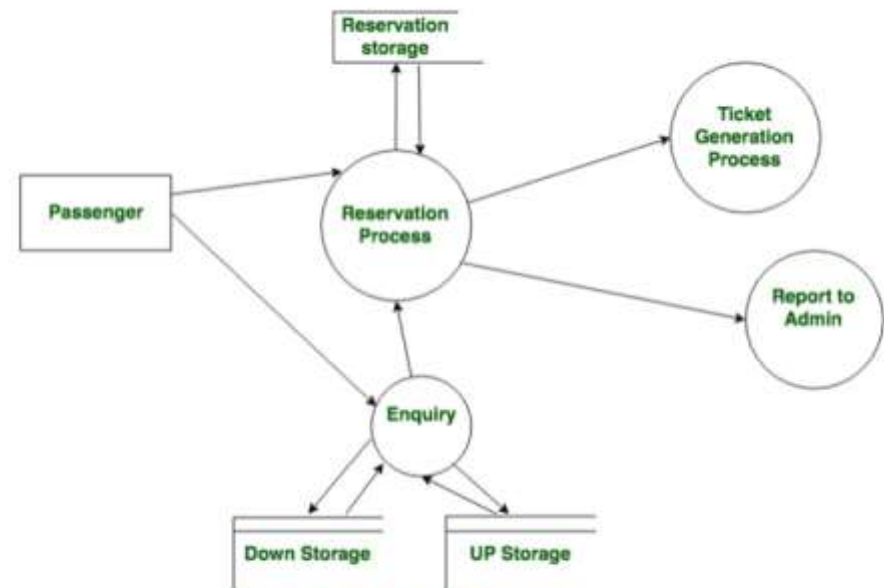


JECRC
UNIVERSITY
BUILD YOUR WORLD

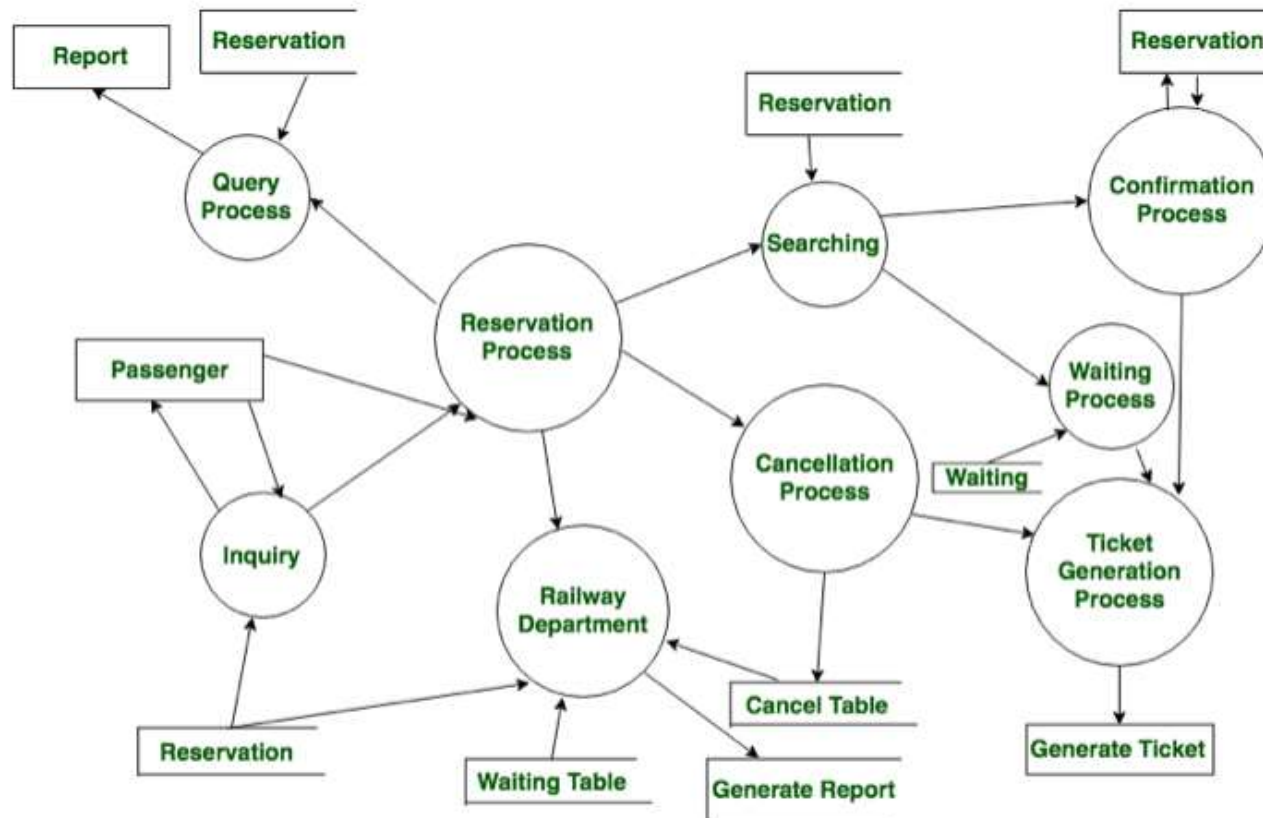




0-LEVEL DFD



1-LEVEL DFD



2-LEVEL DFD

SRS



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

1. “The SRS must correctly define all of the software requirements, but no more.”
2. “The SRS should not describe design, verification, or project management details, except for required design constraints

CHARACTERISTICS OF GOOD SRS



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

1. Unambiguous
2. Complete
3. Verifiable
4. Consistent
5. Modifiable
6. Traceable
7. Usable during the Operation and Maintenance Phase

ER Diagram



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- 1976 proposed by Peter Chen
- ER diagram is widely used in database design

Represent conceptual level of a database system

Describe things and their relationships in high level

Basic Concepts



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Entity set – an abstraction of similar things, e.g. cars, students

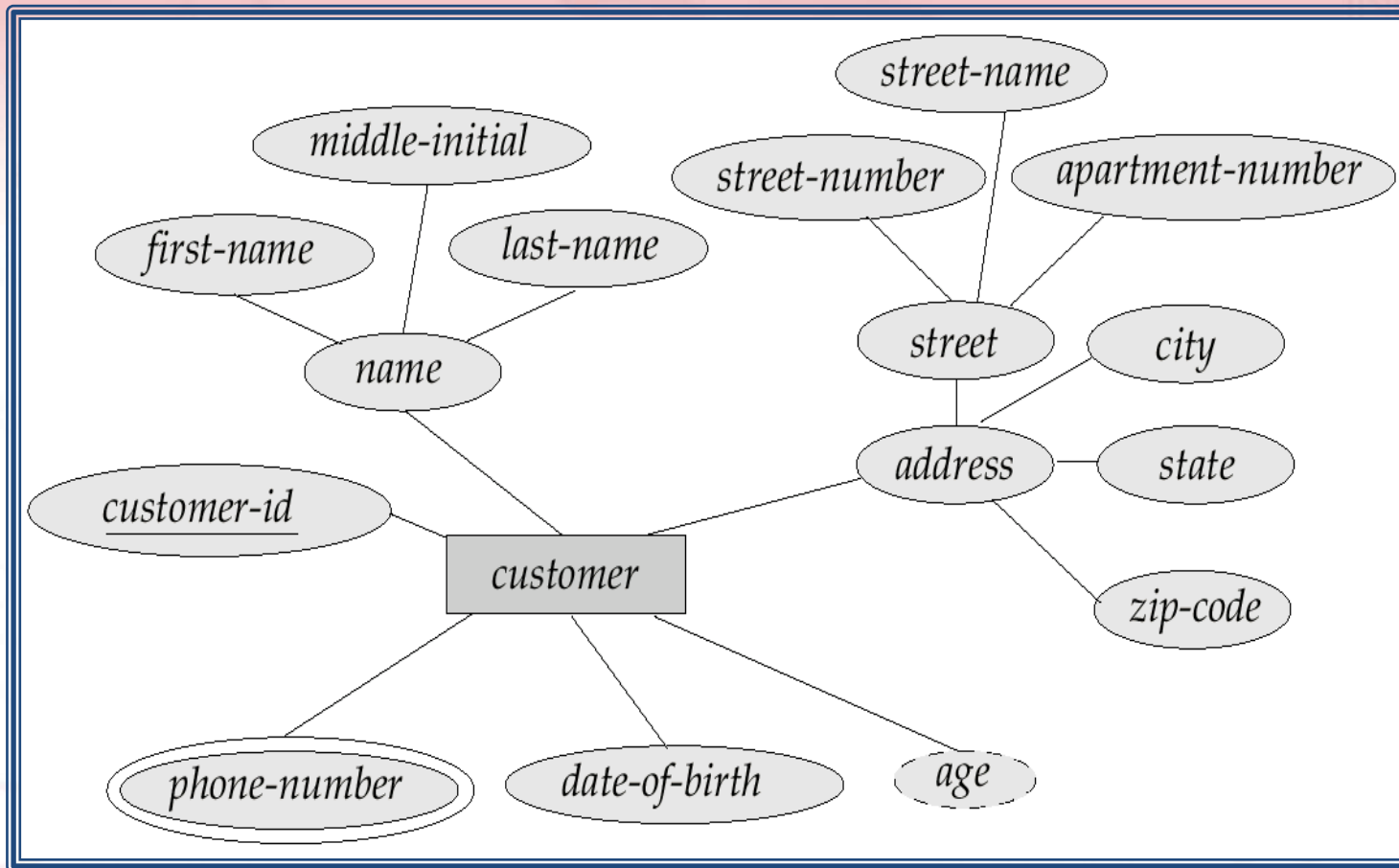
An entity set contains many entities

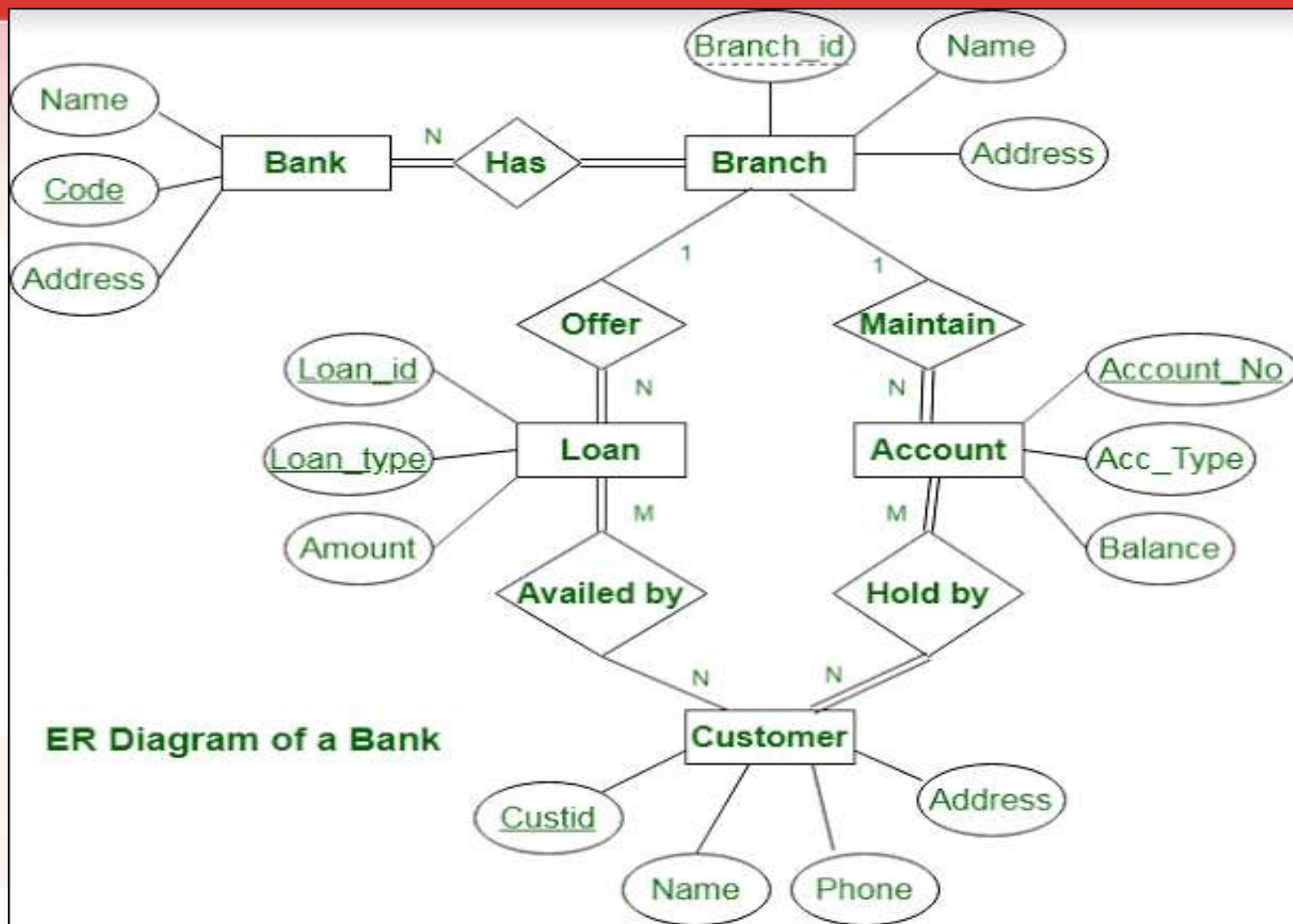
- Attributes: common properties of the entities in a entity sets
- Relationship – specify the relations among entities from two or more entity sets

An Example



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD





Decision Tables



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- A matrix representation of the logic of a decision
- Specifies the possible conditions and the resulting actions
- Best used for complicated decision logic
- Consists of three parts

Condition stubs

- Lists condition relevant to decision

Action stubs

- Actions that result from a given set of conditions

Rules

- Specify which actions are to be followed for a given set of conditions

Cont...



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Indifferent Condition

Condition whose value does not affect which action is taken for two or more rules

- Standard procedure for creating decision tables

Name the condition and values each condition can assume

Name all possible actions that can occur

List all rules

Define the actions for each rule

Simplify the table

Example



JECRC
UNIVERSITY
BUILD YOUR WORLD

	Conditions/ Courses of Action	Rules					
		1	2	3	4	5	6
Condition Stubs	Employee type	S	H	S	H	S	H
	Hours worked	<40	<40	40	40	>40	>40
Action Stubs	Pay base salary	X		X		X	
	Calculate hourly wage		X		X		X
	Calculate overtime						X
	Produce Absence Report		X				

Figure:- Complete decision table for payroll system example

Constructing a Decision Table



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- **PART 1. FRAME THE PROBLEM.**

Identify the conditions (decision criteria). These are the factors that will influence the decision.

- E.g., We want to know the total cost of a student's tuition. What factors are important?

Identify the range of values for each condition or criteria.

- E.g. What are they for each factor identified above?

Identify all possible actions that can occur.

- E.g. What types of calculations would be necessary?

- **PART 2. CREATE THE TABLE.**

Create a table with 4 quadrants.

- Put the conditions in the upper left quadrant. One row per condition.
- Put the actions in the lower left quadrant. One row per action.

List all possible rules.

- Alternate values for first condition. Repeat for all values of second condition. Keep repeating this process for all conditions.
- Put the rules in the upper right quadrant.

Enter actions for each rule

- In the lower right quadrant, determine what, if any, appropriate actions should be taken for each rule.

Reduce table as necessary.

Modularity



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- A concept closely tied to abstraction
- Modularity supports independence of models
- Modules support abstraction in software
- Supports hierarchical structuring of programs
- Modularity enhances design clarity, eases implementation
- Reduces cost of testing, debugging and maintenance
- Cannot simply chop a program into modules to get modularly
- Need some criteria for decomposition

Desired Class/Object Interaction



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Maximize internal interaction (cohesion)
 - easier to understand
 - easier to test
- Minimize external interaction (coupling)
 - can be used independently
 - easier to test
 - easier to replace
 - easier to understand

Characteristics of Good Design



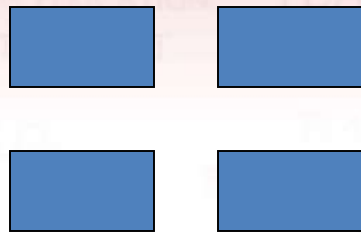
JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Component independence
 - High cohesion
 - Low coupling
- Exception identification and handling
- Fault prevention and fault tolerance

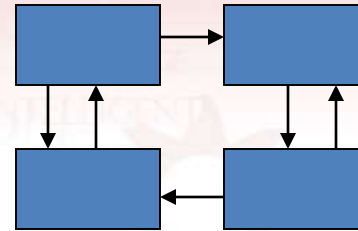
Coupling: Degree of dependence among components



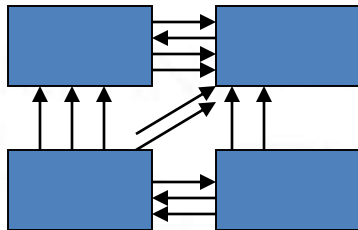
JECRC[™]
UNIVERSITY
BUILD YOUR WORLD



No dependencies



Loosely coupled-some dependencies



Highly coupled-many dependencies

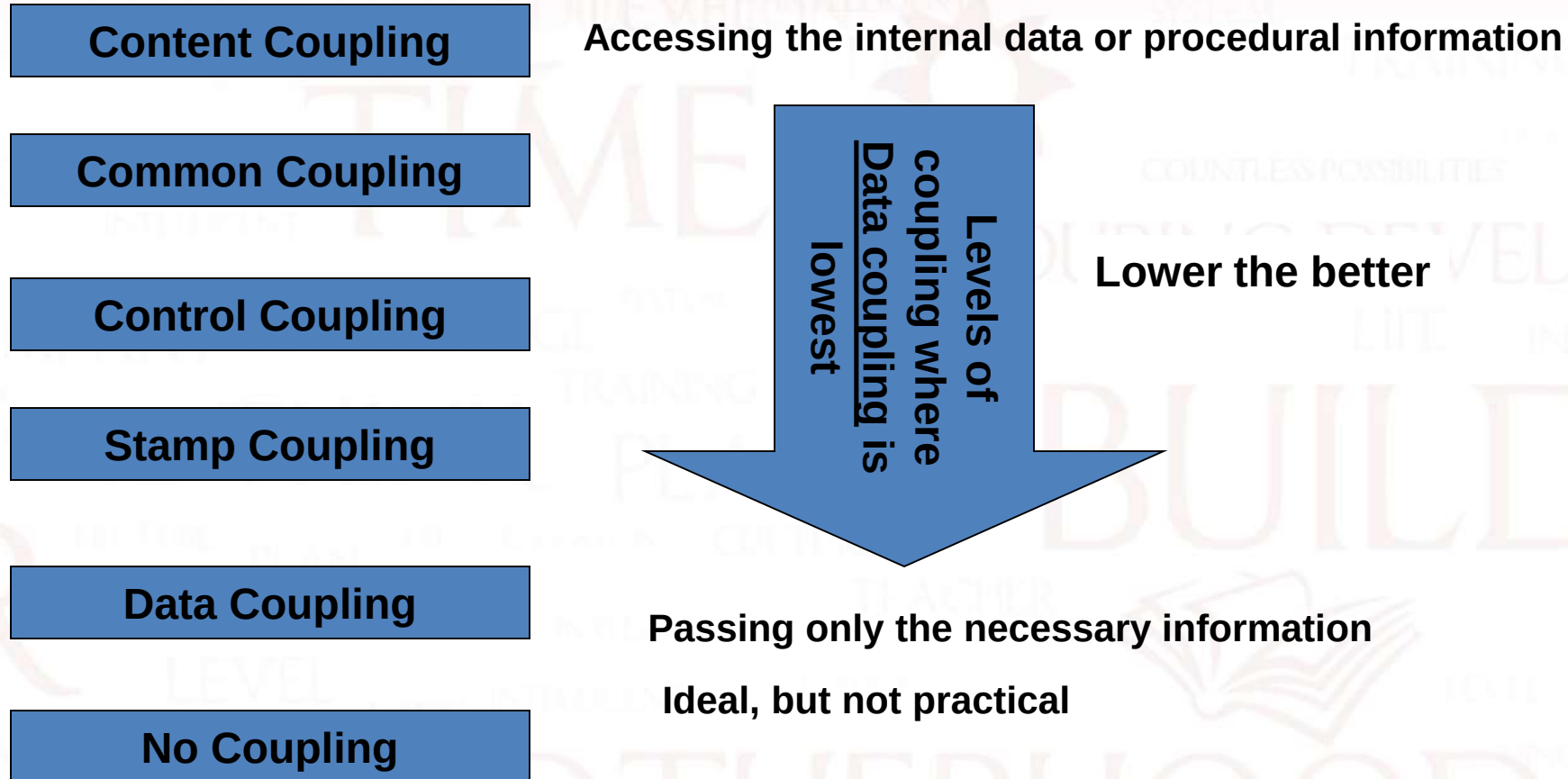
High coupling makes modifying parts of the system difficult, e.g., modifying a component affects all the components to which the component is connected.

Coupling



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Coupling addresses the attribute of “degree of interdependence” between software units, modules or components.



Content Coupling



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: A module directly references the content of another module

module ***p*** modifies a statement of module ***q***

*Module **p** refers to local data of module **q** (in terms of a numerical displacement)*

*Module **p** branches to a local label of module **q***

Content Coupling (cont.)



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Content coupled modules are inextricably interlinked
 - Change to module **p** requires a change to module **q** (including recompilation)
 - Reusing module **p** requires using module **q** also
 - Typically only possible in assembly languages

Common Coupling



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

- Using global variables (i.e., *global coupling*)
- All modules have read/write access to a global data block
- Modules exchange data using the global data block (instead of arguments)
- Single module with write access where all other modules have read access is not common coupling

Common Coupling (cont...)



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

- Have to look at many modules to determine the current state of a variable
- Side effects require looking at all the code in a function to see if there are any global effects
- Changes in one module to the declaration requires changes in all other modules
- Identical list of global variables must be declared for module to be reused
- Module is exposed to more data than is needed

Control Coupling



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: Component passes control parameters to coupled components.
- May be either good or bad, depending on situation.

Bad when component must be aware of internal structure and logic of another module

Good if parameters allow factoring and reuse of functionality

Example



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Acceptable: Module p calls module q and q returns a flag that indicates an error (if any)
- Not Acceptable: Module p calls module q and q returns a flag back to p that says it must output the error “I goofed up”

Stamp Coupling



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: Component passes a data structure to another component that does not have access to the entire structure.
- Requires second component to know how to manipulate the data structure (e.g., needs to know about implementation)
- May be necessary due to efficiency factors: this is a choice made by insightful designer, not lazy programmer.

Data Coupling



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: Two components are data coupled if there are homogeneous data items.
- Every argument is simple argument or data structure in which all elements are used
- Good, if it can be achieved.
- Easy to write contracts for this and modify component independently.

Cohesion



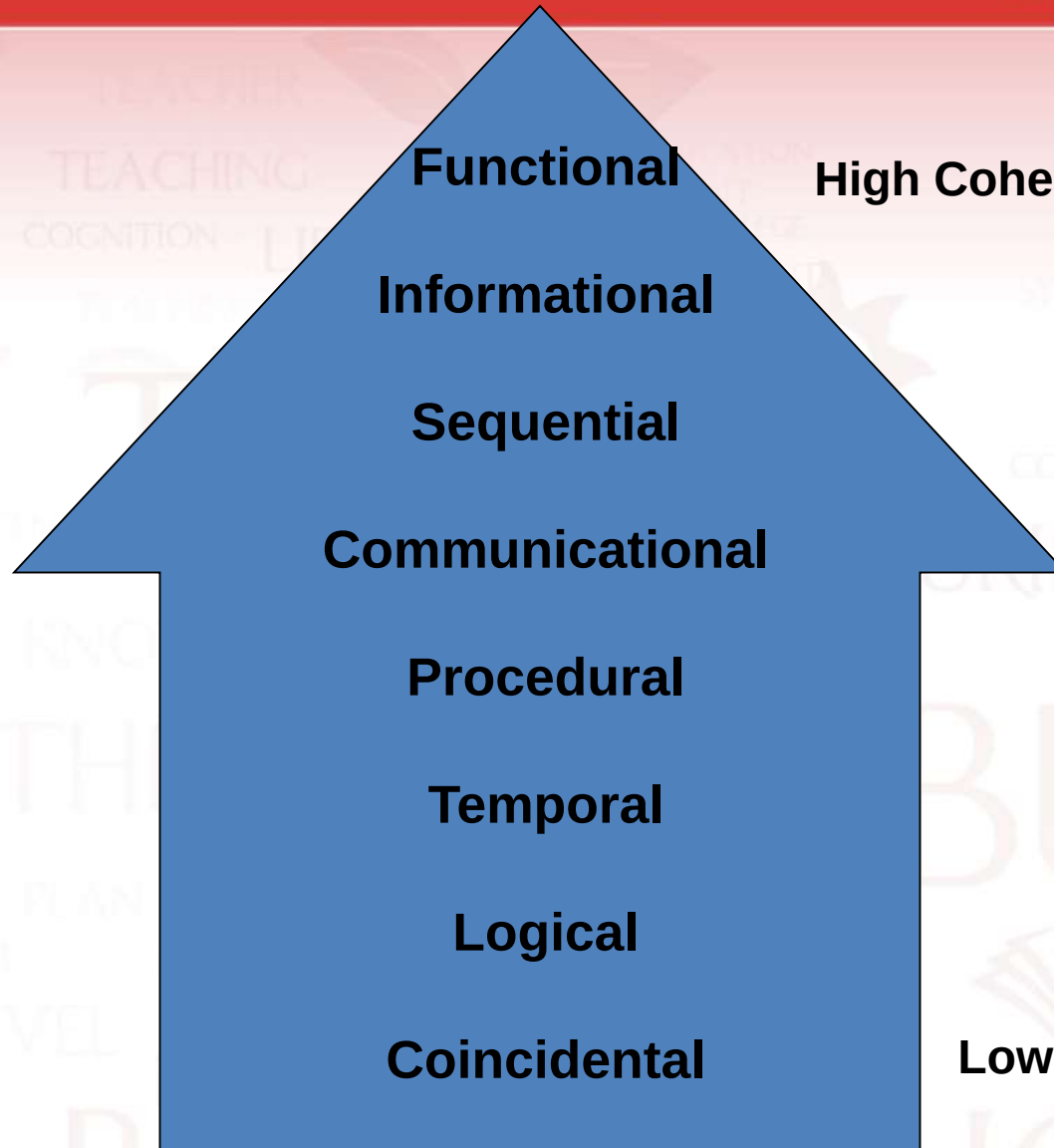
JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: The degree to which all elements of a component are directed towards a single task and all elements directed towards that task are contained in a single component.
- Cohesion of a unit, of a module, of an object, or a component addresses the attribute of “degree of *relatedness*” within that unit, module, object, or component.
- Internal glue with which component is constructed
- All elements of a component are directed toward and essential for performing the same task
- High is good

Range of Cohesion



JECRC
UNIVERSITY
BUILD YOUR WORLD



High Cohesion

Low

Coincidental Cohesion



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: Parts of the component performs multiple, completely unrelated actions
- May be based on factors outside of the design:
 - skillset or interest of developers
 - avoidance of small modules
- No reusability
- Difficult corrective maintenance or enhancement
- Elements needed to achieve some functionality are scattered throughout the system.
- Accidental Worst form
- Example : *an "Utilities" class*

Logical Cohesion



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: Elements of component are related logically and not functionally.
- Several logically related elements are in the same component and one of the elements is selected by the caller.
- May include both high and low-level actions in the same class
- May include unused parameters for certain uses
- Interface is difficult to understand
 - in order to do something you have to wade through a lot of unrelated possible actions
- Example: *grouping all mouse and keyboard input handling routines*

Temporal Cohesion



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: Elements of a component are related by timing.
- Difficult to change because you may have to look at numerous components when a change in a data structure is made.
- Increases chances of regression fault
- Component unlikely to be reusable.
- Often happens in initialization or shutdown
- *Example: a function which is called after catching an exception which closes open files, creates an error log, and notifies the user*

Procedural Cohesion



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: Elements of a component are related only to ensure a particular order of execution.
- Actions are still weakly connected and unlikely to be reusable
- Changes to the ordering of steps or purpose of steps requires changing the module abstraction
- *Example: a function which checks file permissions and then opens the file*

Communicational Cohesion



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: Module performs a series of actions related by a sequence of steps to be followed by the product and all actions are performed on the same data
- Action based on the ordering of steps on all the same data
- Actions are related but still not completely separated
- Module cannot be reused

Sequential Cohesion



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Methods are together in a class because the output from one part is the input to another part like an assembly line
- The output of one component is the input to another.
- Occurs naturally in functional programming languages
- Good situation
- *Example: a function which reads data from a file and processes the data*

Informational Cohesion



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: Module performs a number of actions, each with its own entry point, with independent code for each action, all performed on the same data.
- Different from logical cohesion
 - Each piece of code has single entry and single exit
 - In logical cohesion, actions of module intertwined
- ADT and object-oriented paradigm promote

Functional Cohesion



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Definition: Every essential element to a single computation is contained in the component.
- Every element in the component is essential to the computation.
- Ideal situation.
- *Example: tokenizing a string of XML*

UML



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- Define an easy-to-learn but semantically rich visual modeling language
- Unify the Booch, OMT, and Objectory modeling languages
- Include ideas from other modeling languages
- Incorporate industry best practices
- Address contemporary software development issues
 - scale, distribution, concurrency, executability, etc.
- Provide flexibility for applying different processes
- Enable model interchange and define repository interfaces

What is XML



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- XML stands for eXtensible Markup Language.
- A markup language is used to provide information about a document.
- Tags are added to the document to provide the extra information.
- HTML tags tell a browser how to display the document.
- XML tags give a reader some idea what some of the data means.

What is XML Used For?



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- XML documents are used to transfer data from one place to another often over the Internet.
- XML subsets are designed for particular applications.
- One is RSS (Rich Site Summary or Really Simple Syndication). It is used to send breaking news bulletins from one web site to another.
- A number of fields have their own subsets. These include chemistry, mathematics, and books publishing.
- Most of these subsets are registered with the W3Consortium and are available for anyone's use.

Advantages of XML



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- XML is text (Unicode) based.

Takes up less space.

Can be transmitted efficiently.

- One XML document can be displayed differently in different media.

Html, video, CD, DVD,

You only have to change the XML document in order to change all the rest.

- XML documents can be modularized. Parts can be reused.

Example of an XML Document



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

```
<?xml version="1.0"/>
```

```
<address>
```

```
  <name>Alice Lee</name>
```

```
  <email>alee@aol.com</email>
```

```
  <phone>212-346-1234</phone>
```

```
  <birthday>1985-03-22</birthday>
```

```
</address>
```


Difference Between HTML and XML



JECRC[™]
UNIVERSITY
BUILD YOUR WORLD

- HTML tags have a fixed meaning and browsers know what it is.
- XML tags are different for different applications, and users know what they mean.
- HTML tags are used for display.
- XML tags are used to describe documents and data.



JECRC
UNIVERSITY

BUILD YOUR WORLD
BUILD YOUR WORLD

THANK YOU